Questions

Q1：SIMD与SIMT各是什么？有何区别?

- SIMD（Single Instruction, Multiple Data）
- SIMT（Single Instruction, Multiple Threads）

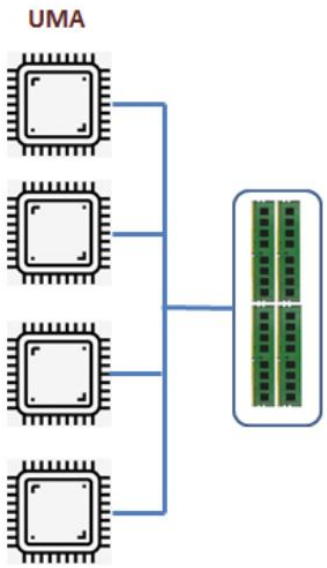
- SIMD：所有处理单元同步执行同一指令，操作不同数据元素，适合规则数据并行处理。
  - 常见于CPU的向量处理单元（如Intel SSE、AVX）和GPU的某些部分

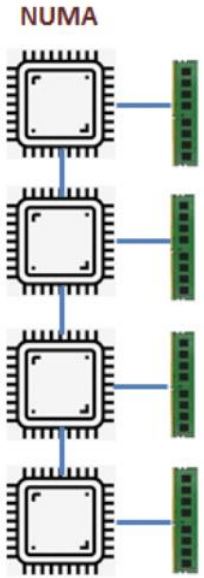
- SIMT：多个线程同时执行同一指令，但每个线程可独立处理不同数据，适合不规则数据并行处理。
  - 主要用于现代GPU，支持大规模并行线程执行

- 两者都旨在提高并行计算效率，SIMT可视为SIMD的扩展，支持更复杂控制流和分支操作

Q2: UMA和NUMA各是什么？有何区别?

- 一致存储访问系统(Uniform Memory Access)
- 非一致存储访问系统(Non-Uniform Memory Access)





Q3: MPP和COW各是什么？有何区别?

- 大规模并行处理器系统(MPP)
- 工作站机群系统(COW)

节点独立性

• **MPP**：节点高度集成，专为并行计算设计，通常没有独立操作系统，依赖全局调度和管理系统。

• **COW**：节点完全独立，各自运行操作系统和应用程序，节点间通过消息传递（如MPI）通信。

网络连接

• **MPP**：使用专用高速互联网络（如InfiniBand），延迟低、带宽高，适合大规模并行计算。

• **COW**：使用标准网络（如以太网），延迟较高、带宽较低，适合松散耦合任务。

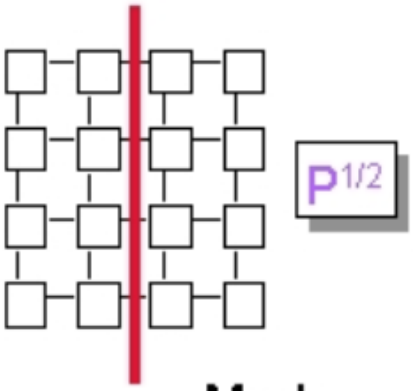
Q4: 互连网络和互联网有何区别与联系?

- **互连网络（Interconnection Network）**：指连接多个计算设备或节点的网络，通常用于高性能计算、数据中心等，强调低延迟和高带宽。
- **互联网（Internet）**：全球范围的公共网络，连接数百万个私有和公共网络，使用TCP/IP协议，提供电子邮件、Web浏览等服务。
- 互连网络可接入互联网，扩展其应用范围，如数据中心通过互连网络连接内部节点，并通过互联网对外服务。

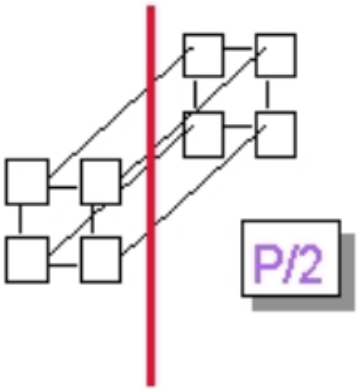
Questions

Q1：什么是等分宽度（ bisection width ）？有何意义?

- A measure of “number of simultaneous communications” or “connectivity”.
- 指将一个网络分成两个大致相等部分时，需要移除的最小边数
- 等分宽度影响处理器间的通信效率，宽度越大，通信效率越高



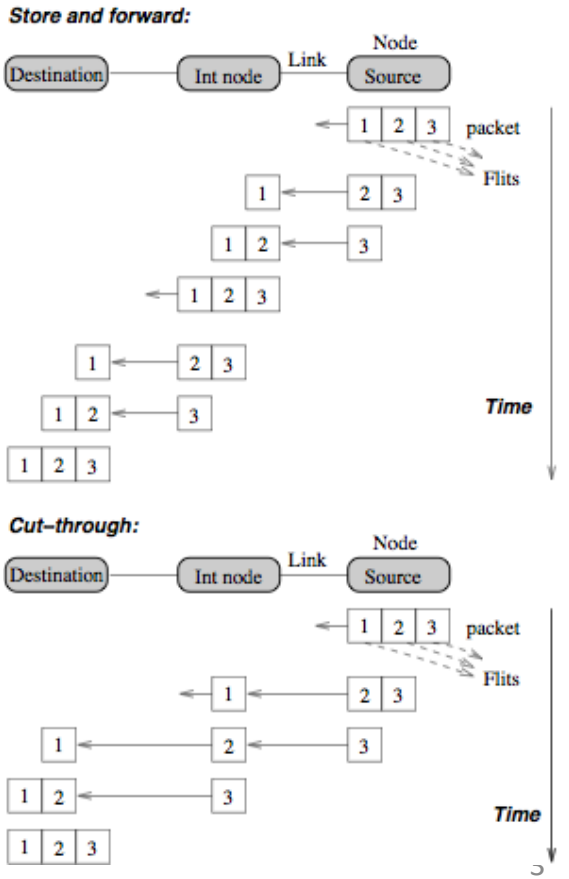
Mesh



Hypercube

Q2: 什么是Cut-Through路由策略？和Store-and-Forward相比有何区别？

- 数据包在到达中间节点时，无需完全接收整个数据包即可开始转发。
- Store-and-Forward：传统方式，节点需接收完整数据包后再转发，延迟较高，但能检测并纠正错误。
- Cut-Through：低延迟，但错误传播风险较高。



Q3: 什么是SPMD编程模式?

- SPMD（Single Program Multiple Data） 是一种并行编程模式，多个处理器同时执行相同的程序，但通过条件语句执行不同的代码。

```
if (I'm thread process i)
    do this;
else
    do that;
```

Q4: 为何多线程并行程序执行时结果存在不确定性（Nondeterminism）？

- **竞争条件（Race Condition）：** 多个线程对共享资源的访问顺序不确定，导致结果依赖于线程执行顺序
- 多个线程同时读写共享数据时，如果没有同步机制（如锁、信号量），可能导致数据不一致。

Q5: 什么是加速比？如何计算？

- **加速比（Speedup）** 是衡量并行计算性能提升的指标，表示并行程序相对于串行程序的执行速度提升倍数。

$$\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

Questions

Q1：什么是可扩展性?

- 可扩展性（scalability）是指系统、网络或软件在需求增加时，能够通过扩展资源（如硬件、软件或网络）来提升性能或容量的能力。

Q2: 什么是强可扩展性？什么是弱可扩展性？

- 强可扩展性指的是在保持问题规模不变的情况下，增加处理器数量时，算法或系统的执行时间能够相应减少。
- 换句话说，如果一个问题的大小固定，当我们将处理器数量翻倍时，理论上我们希望执行时间至少减半。
- 弱可扩展性则是在处理器数量增加的同时，也相应地增加问题规模，使得每个处理器处理的数据量大致相同。
- 在这种情况下，我们期望算法或系统的效率（即单位时间内解决的问题规模）不随处理器数量的增加而下降。

Q5: 强可扩展性和弱可扩展性有何区别?

区别:

- 强可扩展性：固定问题规模，增加处理器数量，目标是减少执行时间。
- 弱可扩展性：同时增加问题规模和处理器数量，目标是保持或提升每单位时间的计算效率。

Q4:并行程序设计中的Foster方法论是什么?

- Foster方法论是一种用于设计和分析并行算法的系统方法。包括以下主要步骤:
  - 划分（Partitioning）：将问题划分为可以独立解决的较小任务。
  - 通信（Communication）：确定哪些数据需要在不同的处理器之间进行交换。
  - 聚合（Agglomeration）：将较小的任务组合成较大的任务，以减少通信开销和提高计算效率。
  - 映射（Mapping）：将任务分配到具体的处理器上。

Q5: MPI全称是什么？它是一种编程语言吗？是面向哪种体系结构的？

- MPI的全称是Message Passing Interface，即消息传递接口。
- MPI是一种用于编写并行计算机程序的标准库，主要用于多进程间的通信和数据交换。
- MPI不是一种编程语言，而是一组函数库，可以被多种编程语言调用，最常见的是C、C++和Fortran。
- MPI适用于分布式内存的多处理器系统。

Questions

Q1：什么是集合通信？有何特点和用处？

- **集合通信**是 MPI中的一种通信模式，它涉及一组进程（通常是通信域中的所有进程）之间的协同操作。
- **集合通信的特点**
  - **全局性**：集合通信涉及通信域中的所有进程，而不是单独的两个进程。
  - **同步性**：所有参与的进程必须同时调用集合通信函数，否则会导致程序挂起或错误。
  - **高效性**：集合通信通常经过优化，能够利用底层硬件和网络的特性，提高通信效率。
- **集合通信的用途**：数据分发与收集、数据规约与聚合、进程同步、高效通信

Q2：集合通信和点到点通信有何区别?

特性	点对点通信	集合通信
参与进程	两个进程	通信域中的所有进程
同步性	不需要全局同步	隐式同步
通信模式	一对一	一对多、多对一或多对多
典型操作	`MPI_Send`、`MPI_Recv`	`MPI_Bcast`、`MPI_Reduce`
性能	适用于局部通信	适用于全局通信，经过优化
代码复杂性	较高	较低
适用场景	不规则通信、精细控制	规则通信、全局协同操作

Q3：MPI_Reduce是干什么的?

- **MPI_Reduce （MPI规约操作）**：将通信域中所有进程的数据通过某种操作（如求和、求最大值等）合并为一个结果，并存储在根进程中。

- 示例：所有进程发送一个数值，根进程计算这些数值的总和。

Q4：所有进程必须按照相同的顺序调用集合通信函数吗？

- 是的。
- 集合通信操作是全局同步的，所有进程必须按照相同的顺序调用集合通信函数。
- MPI 通过调用顺序来匹配集合通信操作，确保每个进程的发送和接收操作能够正确对应。

Q5：Wall Clock Time与CPU Time有何区别?

- **Wall Clock Time**：程序运行的实际时间，包括所有计算、等待和 I/O 时间。
- **CPU Time**：程序实际使用 CPU 进行计算的时间，不包括等待时间。
- 示例：
  - 假设一个程序运行了 10 秒的 Wall Clock Time，但其中只有 4 秒是 CPU 计算时间，其余 6 秒可能是等待 I/O 或资源的时间。

Questions

Q1：MPI derived datatypes 有什么用处?

- MPI派生数据类型（Derived Datatypes）用于定义复杂的数据结构，以便在MPI进程之间高效地传输非连续或异构的数据。

- 派生数据类型的主要用途是简化通信操作，提高代码的可读性和性能。

Q2：MPI_Allgather可以实现什么功能?

- MPI_Allgather是一个MPI集合通信函数，用于将每个进程的数据收集到所有进程中，实现了全收集（All-to-All）的通信模式

Q3：MPI_Allgather和MPI_Gather有何不同?

- 与MPI_Gather不同，MPI_Allgather 不仅将数据收集到根进程，还会将收集到的数据分发到所有进程

Q4：MPI_Scatter可以实现什么功能?

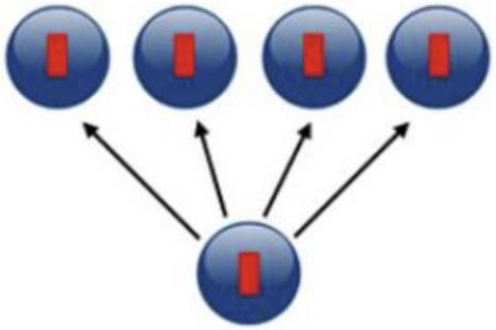
- MPI_Scatter用于将根进程（Root Process）的数据分散（Scatter）到所有进程中

Q5：MPI_Scatter和MPI_Bcast有何不同?

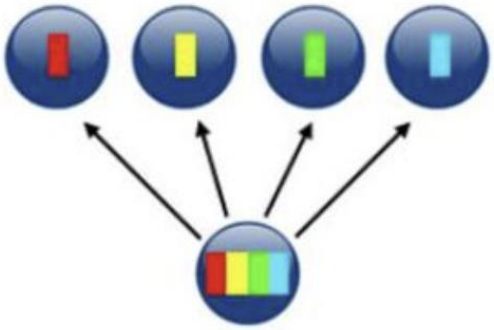
- 与 MPI_Bcast 不同，MPI_Scatter 不是简单地将相同的数据发送给所有进程，而是将数据的不同部分分别发送给不同的进程。

Questions

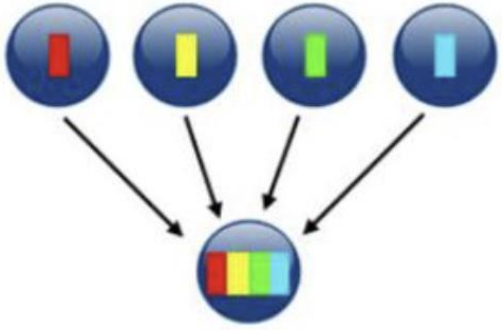
Q0：MPI_Bcast、MPI_Scatter、MPI_Gather、MPI_Reduce这几个集合函数有何区别?



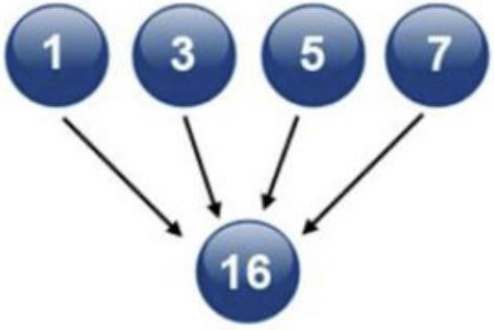
broadcast



scatter



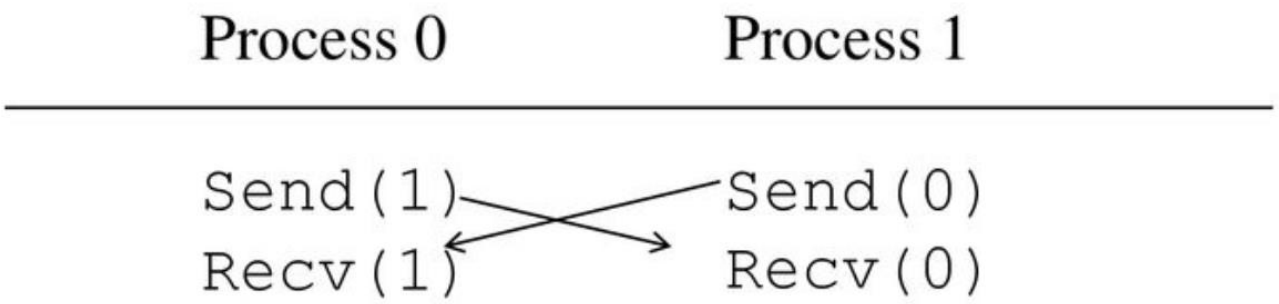
gather



reduction

Q1：什么是MPI Safety问题？举个例子

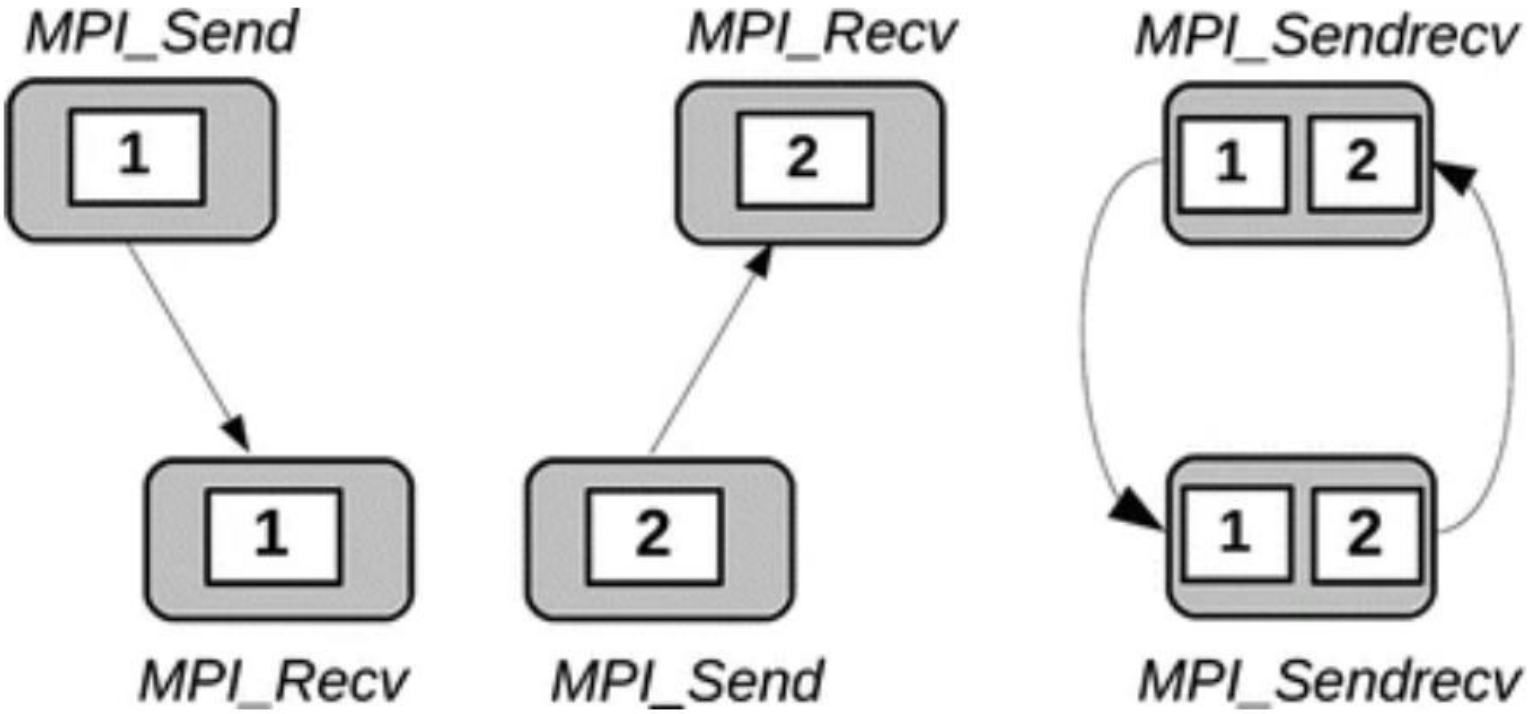
- MPI Safety问题是指在MPI编程中，由于MPI的实现或使用不当，导致程序在多进程环境下出现**未定义行为、数据竞争、死锁**或其他错误的现象。
- 例子：MPI 通信操作（如 MPI_Send 和 MPI_Recv）的顺序可能导致**死锁**



Q2：调用MPI_SendRecv和直接调用MPI_Send、MPI_Recv有何区别?

- MPI_SendRecv：
  - 在一个调用中同时完成发送和接收操作
  - 发送和接收操作是原子的，避免了死锁风险

- MPI_Send 和 MPI_Recv：
  - 需要分别调用发送和接收函数
  - 如果调用顺序不当，可能导致死锁


MPI_Send
1
1
MPI_Recv
MPI_Recv
2
2
MPI_Send
MPI_Sendrecv
1
2
1
2
MPI_Sendrecv

Q3：什么是奇偶转置排序（Odd-even transposition sort）？

- **奇偶转置排序**是一种简单且易于并行化的排序算法
- Even phases, compare swaps:

$$(a[0],a[1]),(a[2],a[3]),(a[4],a[5]),\ldots$$

- Odd phases, compare swaps:

$$(a[1],a[2]),(a[3],a[4]),(a[5],a[6]),\ldots.$$

Q4：手动实现下面序列的奇偶转置排序

[5, 2, 9, 1, 5, 6]

Q5：为何奇偶转置排序相对冒泡排序更加易于并行化？

- **奇偶转置排序：**
  - 所有比较-交换操作是独立的。
  - 这些操作可以同时进行，无需等待其他操作的完成。

- **冒泡排序：**
  - 每一轮的比较-交换操作是顺序依赖的，即每次比较-交换都需要等待前一次操作的结果。
  - 无法同时进行多个比较-交换操作。

Questions

Q1: 为何用2个线程计算pi的值反而不准?

	n			
	10^5	10^6	10^7	10^8
π	3.14159	3.141593	3.1415927	3.14159265
1 Thread	3.14158	3.141592	3.1415926	3.14159264
2 Threads	3.14158	3.141480	3.1413692	3.14164686

- **竞争条件（Race Condition）的问题，合并各线程的结果时出现了问题**
  - 如果两个线程共享了同一个变量，而多个线程同时对这个变量进行写操作，导致竞争条件

Q2: 线程之间是如何实现数据传递的?

- **线程通过访问共享变量或数据结构（如全局变量、堆内存、队列等）传递数据**

Q3: Busy-waiting与Mutex在实现多线程对临界区访问时有什么区别？哪个效率更高？

- **Busy-waiting：低延迟但高CPU占用，适合极短操作。**
  - 避免了内核态切换的开销
  - 内核态切换是指CPU在执行程序时，从用户态切换到内核态（或反之）的过程

- **Mutex：高延迟但节省CPU，适合通用场景。**

Q4: Mutex与Semaphore有什么区别？简单说一下。

特性	Mutex（互斥锁）	Semaphore（信号量）
用途	保护共享资源的**独占访问**（临界区）	控制**资源数量**或线程/进程的**同步**
持有者	必须由**加锁的线程解锁**（所有权概念）	可由**任意线程释放**（无所有权）
计数	只能是 **0（未锁）或 1（已锁）**	可以是 **任意非负整数**（允许多个访问）
线程阻塞	竞争失败的线程会**阻塞**	如果计数≤0，线程会**阻塞**
典型场景	保护全局变量、数据结构	限流（如数据库连接池）、生产者-消费者

Questions

Q1: pthreads中的条件变量（condition variable）有什么用途?

条件变量是一种线程同步机制，用于在多个线程之间高效地等待某个条件的成立。

让线程在某个条件不满足时主动阻塞，并在条件满足时被唤醒继续执行。

Q2: 为何pthreads中的条件变量（condition variable）总是与一个mutex相关联?

- **线程在等待某个条件成立时，需要先检查条件是否满足，但检查条件本身必须是原子操作**
- **通过互斥锁包裹条件的检查和修改，确保“检查条件”和“进入等待”是原子的，避免竞争条件**

Q2: 为何pthreads中的条件变量（condition variable）总是与一个mutex相关联?

条件变量的等待操作 pthread_cond_wait() 实际上会执行以下三步：

- 释放关联的 Mutex（让其他线程可以修改共享数据）。
- 阻塞线程，等待条件变量的信号。
- 被唤醒后，重新获取 Mutex（继续操作共享数据）。

如果没有Mutex：线程无法安全地释放和重新获取锁，可能导致共享数据被破坏。

Q3: 如何自己用pthreads实现一个Barrier（栅栏）？

有哪些方法?

方法1：通过busy-waiting + 互斥锁来实现

方法2：通过使用信号量（Semaphore）来实现

方法3：通过使用互斥锁 + 条件变量来实现

- 使用一个计数器记录已到达的线程数。
- 用互斥锁保护计数器。
- 当计数器未达到总线程数时，线程通过条件变量等待。
- 当最后一个线程到达时，唤醒所有线程。

Implementing a barrier with condition variables

```
/* Shared */
int counter = 0;
pthread_mutex_t mutex;
pthread_cond_t cond_var;
. . .
void* Thread_work(. . .) {
    . . .
    /* Barrier */
    pthread_mutex_lock(&mutex);
    counter++;
    if (counter == thread_count) {
        counter = 0;
        pthread_cond_broadcast(&cond_var);
    } else {
        while (pthread_cond_wait(&cond_var, &mutex) != 0);
    }
    pthread_mutex_unlock(&mutex);
    . . .
}
```

pthread_mutex_unlock(&mutex);
wait_on_signal(&cond_var);
pthread_mutex_lock(&mutex)



Q4: 多个线程同时并行操作一个链表（如执行member、delete、insert操作）会出现什么问题?

会引发一系列线程安全问题，主要包括竞争条件（Race Condition）、数据不一致性、甚至程序崩溃

- **插入（Insert） + 删除（Delete）竞争：**
线程A准备在节点 X 后插入新节点 Y，但线程B同时删除了 X，导致 Y 插入到无效位置或链表断裂。

- **删除（Delete） + 查询（Member）竞争：**
线程A正在读取节点 X 的数据，线程B同时删除 X，导致线程A访问已释放的内存（悬垂指针）

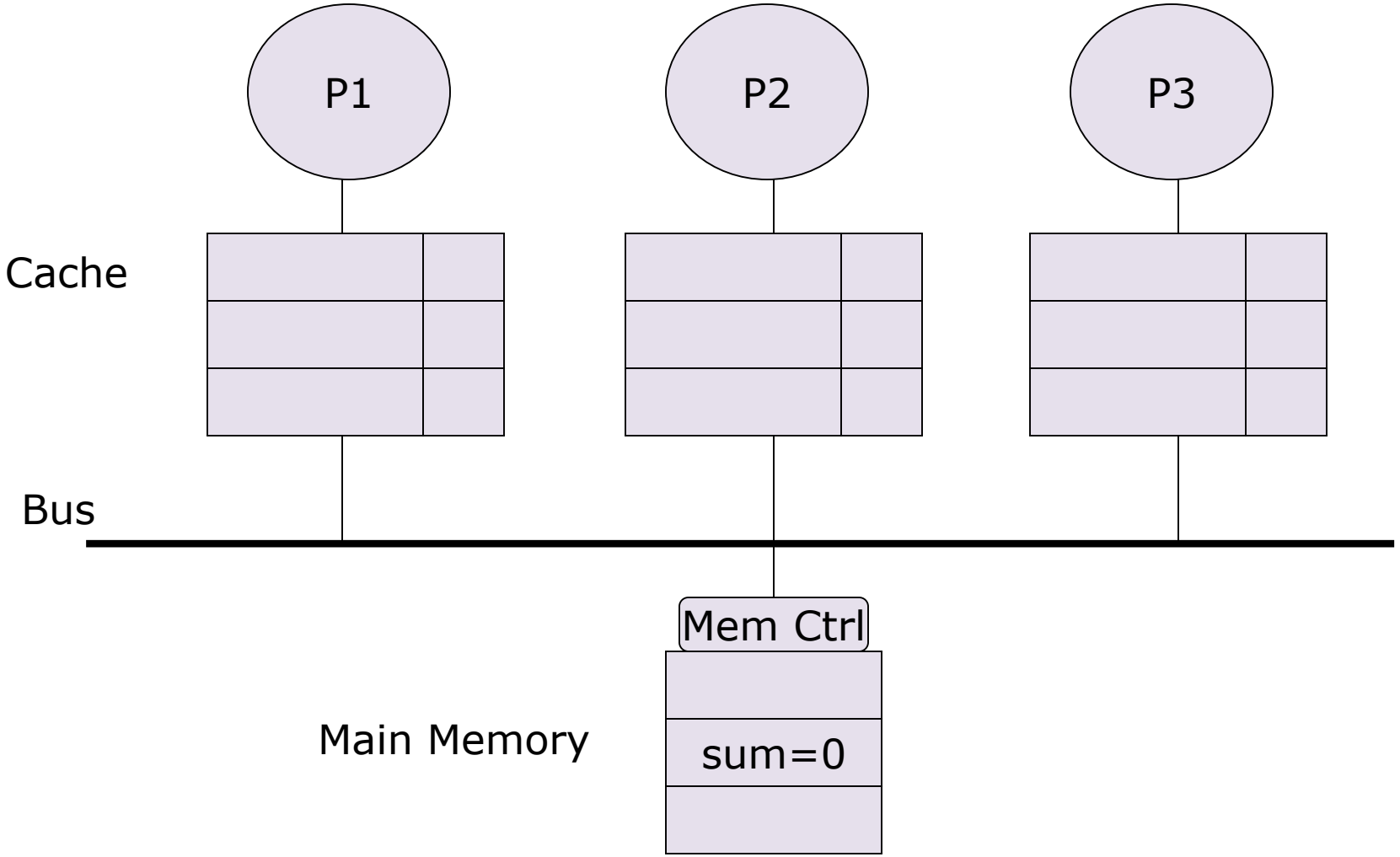
Questions

Q1: 什么是Cache coherence（缓存一致性）问题？可否举个例子。

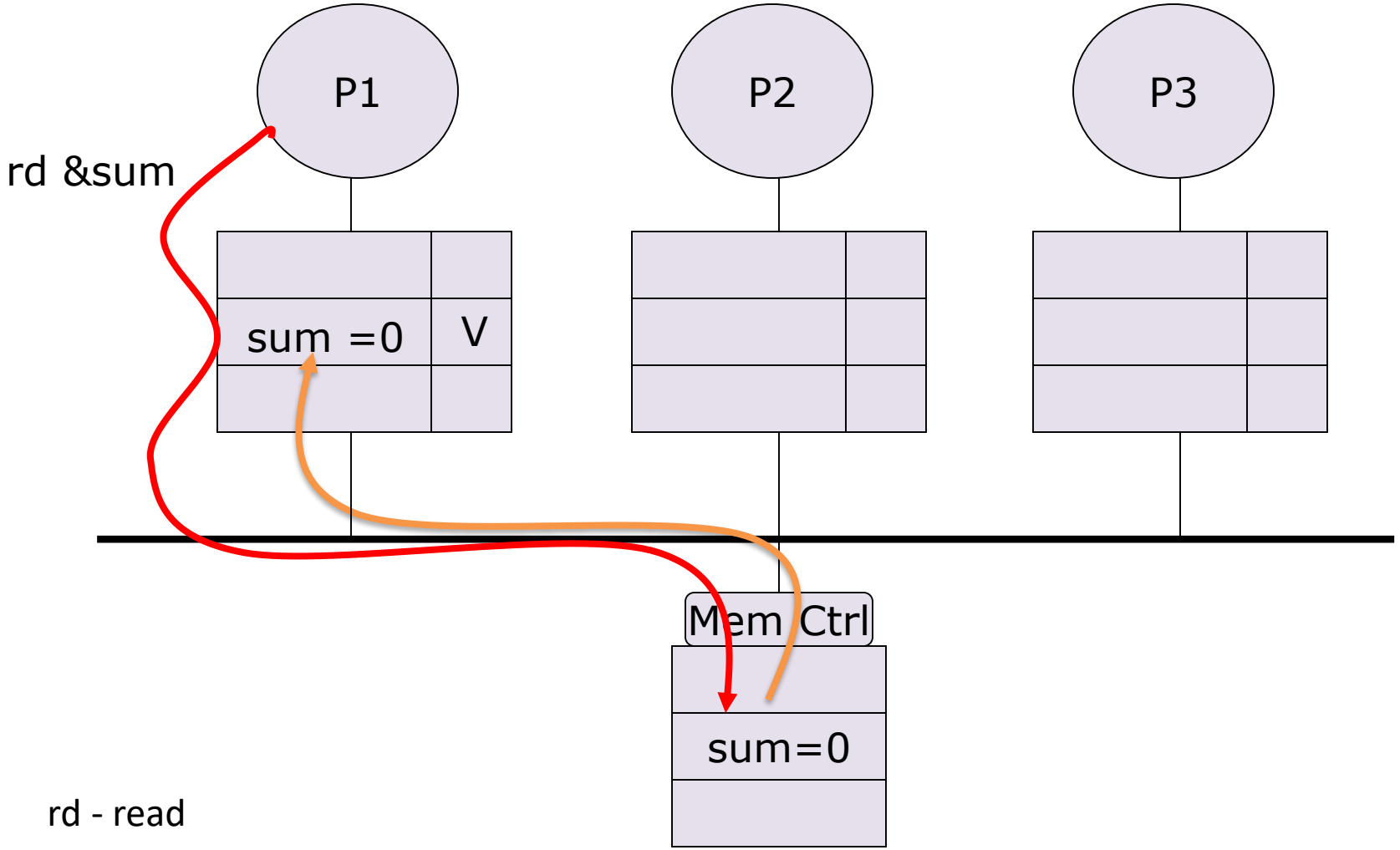
- **缓存一致性**是指在多处理器系统中，当多个处理器核心拥有自己的缓存时，如何确保它们共享数据的一致性。

- 当不同处理器缓存了同一内存位置的数据副本时，对这些副本的更新必须传播到所有其他缓存，以保持数据一致性。

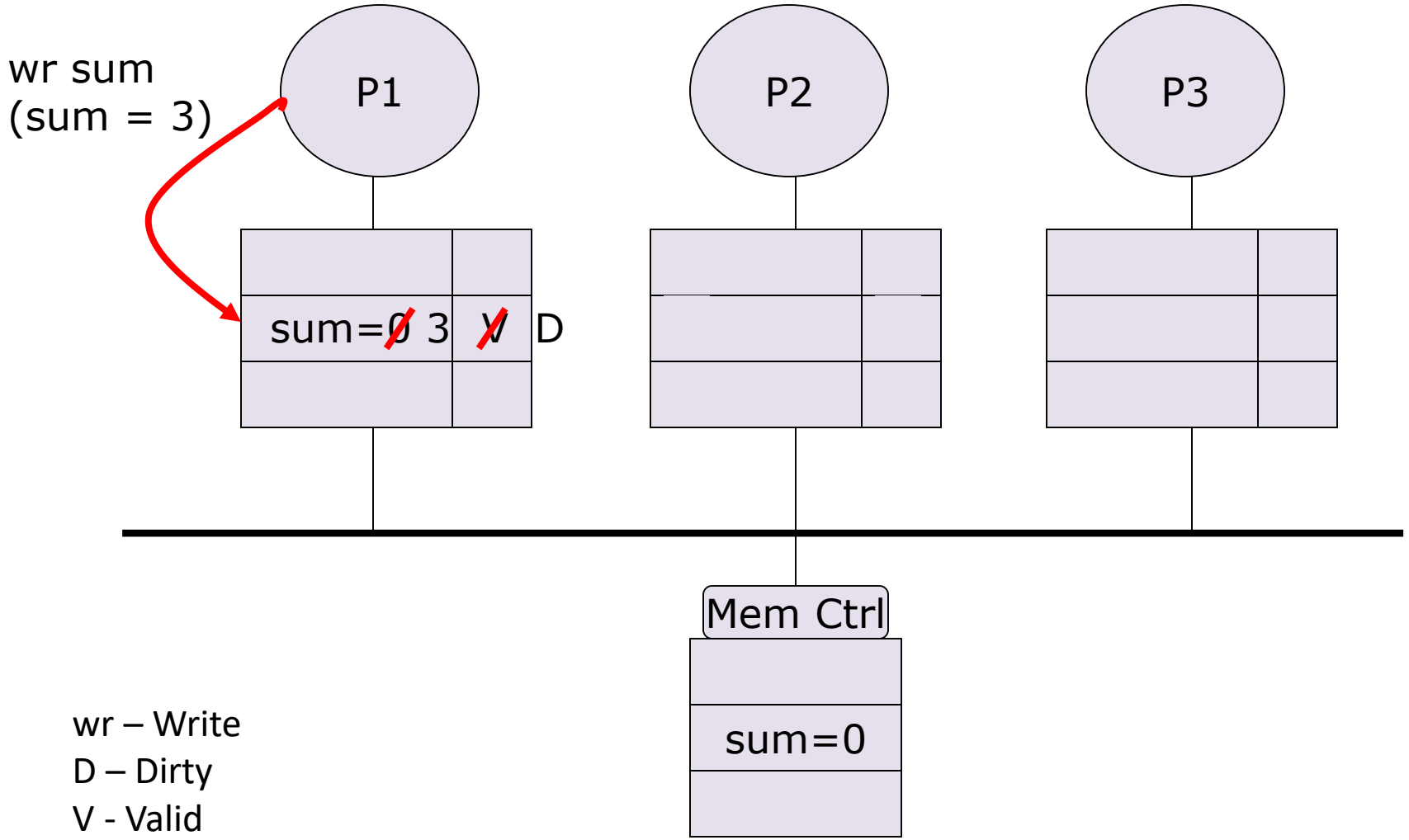
缓存一致性问题举例



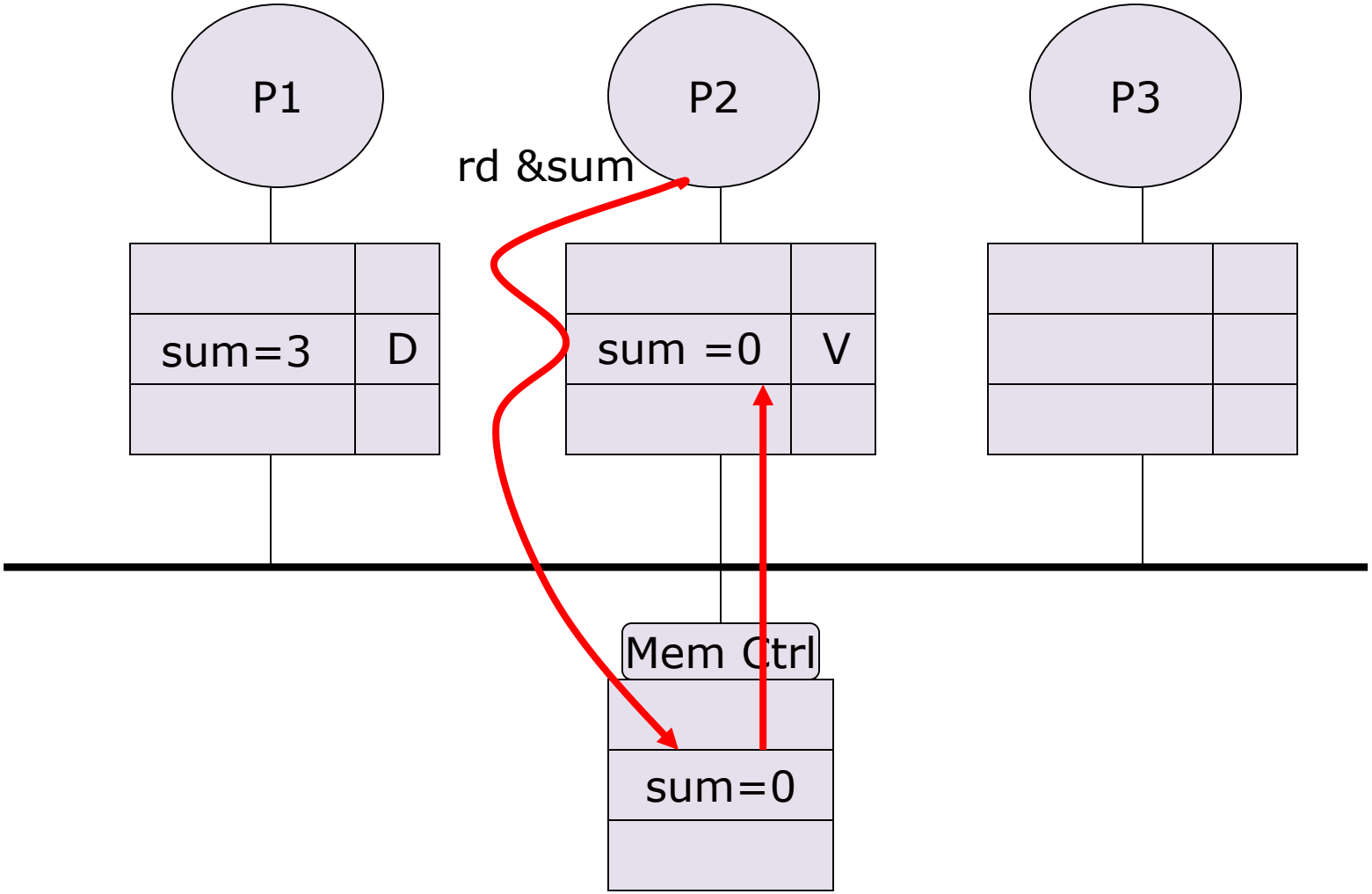
缓存一致性问题举例



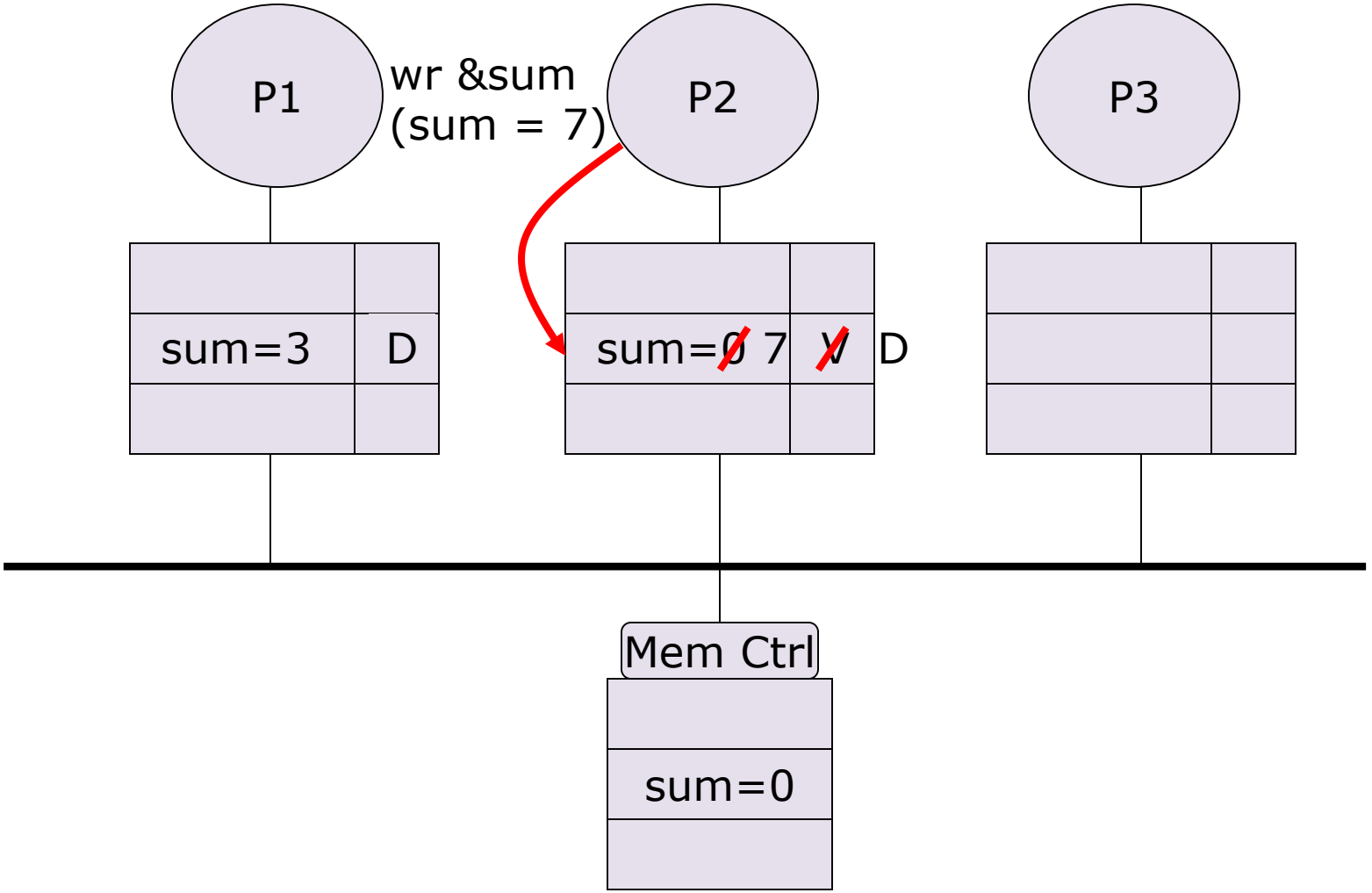
缓存一致性问题举例


wr sum
(sum = 3)
P1
P2
P3
sum=0 3 V D
Mem Ctrl
sum=0
wr – Write
D – Dirty
V - Valid

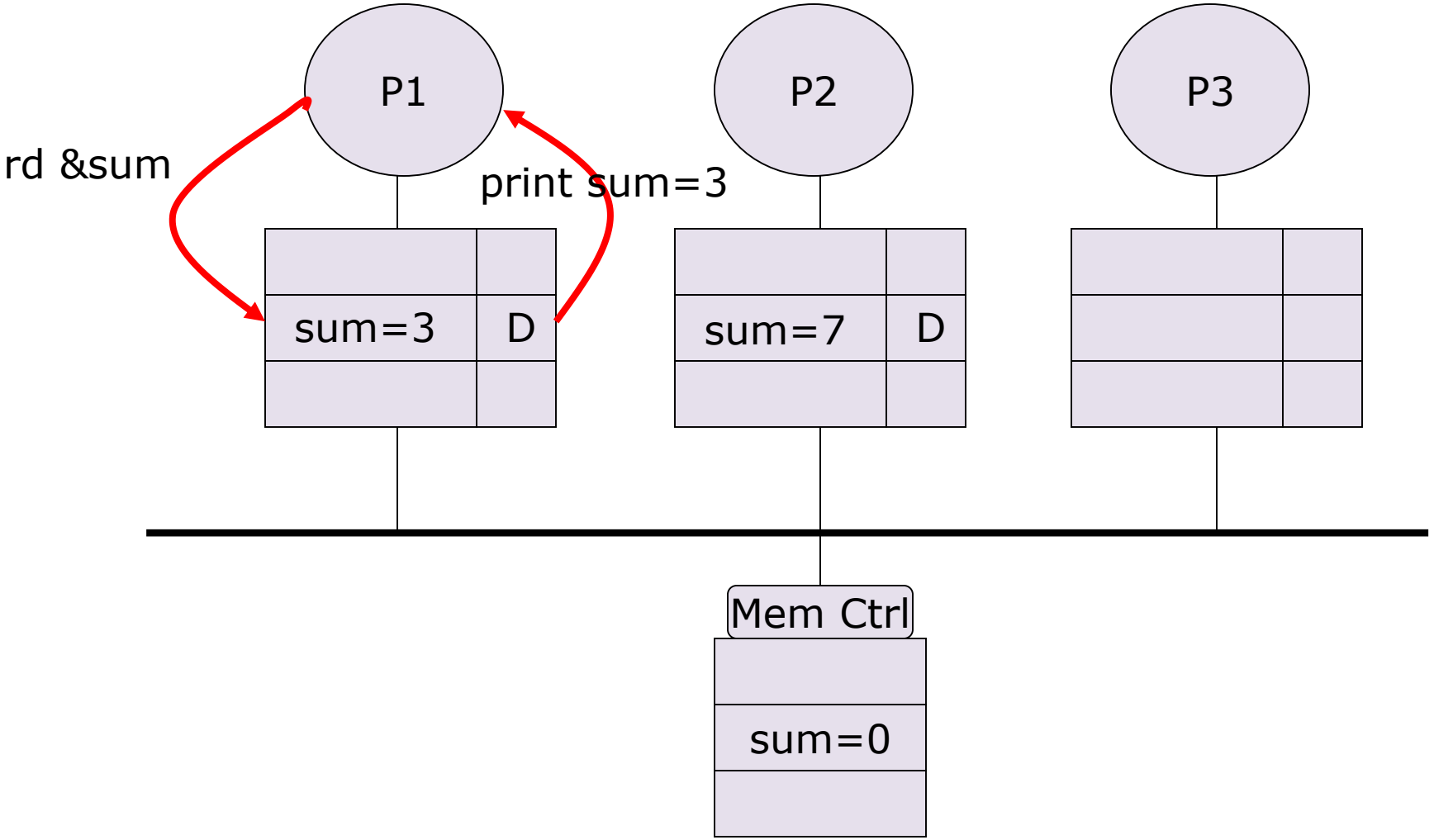
缓存一致性问题举例



缓存一致性问题举例



缓存一致性问题举例



Q2：什么是False Sharing （伪共享）问题？可否举个例子。

- **False Sharing（伪共享）发生在多个线程同时访问同一缓存行（Cache Line）中的不同变量时，导致不必要的缓存同步开销，从而降低程序性能。**

- **False Sharing 的原理：**
  - CPU 缓存以固定大小的块（通常 64 字节）加载数据，即使只修改其中一个变量，整个缓存行也会被标记为"脏"（Dirty），并触发缓存一致性协议的同步操作。
  - **伪共享：**如果两个线程分别修改同一个缓存行中的不同变量，即使它们没有真正的数据依赖关系，CPU 仍然会强制同步缓存，导致性能下降。

Q2：什么是False Sharing （伪共享）问题？可否举个例子。

```
struct Data {
    int x;  // Thread A 频繁修改 x
    int y;  // Thread B 频繁修改 y
};
Data data;  // x 和 y 在同一个缓存行
```

- Thread A 不断修改 data.x，导致 CPU 缓存行被标记为 修改 (Modified)
- Thread B 修改 data.y 时，由于 y 和 x 在同一个缓存行，CPU 必须：
  1. 使 Thread A 的缓存行失效 (Invalidate)
  2. 从内存重新加载缓存行 (Read)
  3. 修改 y 并标记为"修改" (Modified)
- 这样，即使 x 和 y 没有数据依赖关系，缓存行仍然不断在 CPU 核心之间同步，导致性能下降。

Q3: 什么是Thread Safety（线程安全）问题？

如何解决线程安全性问题？

- **线程安全（Thread Safety） 指在多线程环境下，程序或数据结构能够正确、一致地工作，而不会因并发访问导致数据损坏、逻辑错误或不可预测的行为。**

- **解决方法：**
  - **选用线程安全的函数**
  - **互斥锁：确保同一时间只有一个线程能访问共享资源**
  - **原子操作：确保操作不可分割**
  - **局部存储：让每个线程拥有独立的变量副本，避免共享数据**
  - **……**

Q4: OpenMP与pthreads都是多线程编程，它们有何区别?

特性	OpenMP	pthreads
线程创建	自动(由编译器/运行时处理)	手动(pthread_create)
线程同步	内置同步结构	需显式使用互斥锁/条件变量
负载均衡	自动调度	手动实现
可移植性	跨平台(需编译器支持)	UNIX-like系统
学习曲线	较简单	较复杂
控制粒度	较粗	精细
调试难度	较容易	较困难

Questions

OpenMP reduction子句的变量初始值

- 在 OpenMP 的 reduction 子句中，每个线程的私有变量副本的初始值由归约操作符的类型决定

- 初始值规则
  - 加法 (+)：初始值为 0
  - 乘法 (*)：初始值为 1
  - 逻辑与 (&&)：初始值为 1（即 true）
  - 逻辑或 (||)：初始值为 0（即 false）
  - 最大值 (max)：初始值为该数据类型的最小可能值（如 INT_MIN 对于 int 类型）
  - 最小值 (min)：初始值为该数据类型的最大可能值（如 INT_MAX 对于 int 类型）

- 假设使用 reduction(+:sum)，每个线程的私有 sum 会初始化为 0
- 在并行区域内，各线程操作自己的副本
- 退出并行区域时，所有线程的副本通过加法合并，结果再与主线程的原始值相加。

```c
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>

int main(int argc,char *argv[])
{
    int sum=10; //主线程的原始值

    int thrdCnt=strtol(argv[1],NULL,10);

    #pragma omp parallel num_threads(thrdCnt) reduction(+:sum)
    {
        sum+=1;  //每个线程的私有sum初始为0，  执行后变为1
    }

    printf("Final sum = %d\n", sum);  //输出为10+4=14，  假设4个线程

    return 0;

}
```

- 执行结果：
  - 每个线程的私有 sum 初始化为 0
  - 并行结束后，各线程的 sum 相加为 4（假设 4 个线程）
  - 主线程的原始值 10 与归约结果 4 相加，最终 sum = 14

操作符与初始值的对应关系

- 加法（+）：
  - 私有副本初始为 0，最终结果为 原始值 + 各线程和。
- 乘法（*）：
  - 私有副本初始为 1，最终结果为 原始值 * 各线程乘积。
- 最大值 (max):
  - 私有副本初始为 INT_MIN，最终结果为 max(原始值, 各线程最大值)

Q1: OpenMP中是否也存在临界区的问题?

如何解决?

```
# pragma omp critical
  global_result += my_result ;
```

Q2：在OpenMP中，一个变量的范围指什么？

In OpenMP, the scope of a variable refers to the set of threads that can access the variable in a parallel block.

Q4: 直接在一个循环体（loop）外面加上

```
# pragma omp parallel for
```

来实现循环体并行化是否总是可行的？会不会出错？

```
fibo[ 0 ] = fibo[ 1 ] = 1;
# pragma omp parallel for num_threads(2)
for (i = 2; i < n; i++)
    fibo[ i ] = fibo[ i – 1 ] + fibo[ i – 2 ];
```

1 1 2 3 5 8 13 21 34 55

this is correct

but sometimes we get this

1 1 2 3 5 8 0 0 0 0

- OpenMP compilers don't check for dependencies among iterations in a loop that's being parallelized with a parallel for directive.

- A loop in which the results of one or more iterations depend on other iterations cannot be correctly parallelized by OpenMP.

Questions

Q1: #pragma omp parallel for 编译指令和 #pragma omp for 编译指令有何区别? 哪个效率更高?

#pragma omp for tells OpenMP to parallelize the for loop with existing team of threads

Q2：为何要对循环体进行调度（把iterations分配到不同线程）？有哪些调度类型?

schedule (type , chunksize)

Type can be:

- **static**: the iterations can be assigned to the threads before the loop is executed.
- **dynamic or guided**: the iterations are assigned to the threads while the loop is executing.
- **auto**: the compiler and/or the run-time system determine the schedule.
- **runtime**: the schedule is determined at run-time.

Q3: Dynamic schedule与Guided schedule有何区别?

- Each thread also executes a chunk, and when a thread finishes a chunk, it requests another one.
- However, in a guided schedule, as chunks are completed, the size of the new chunks decreases.

Q4：OpenMP提供了哪些机制来保障对于临界区访问的互斥?

- OpenMP provides several mechanisms for insuring mutual exclusion in critical sections.
  - Critical directives
  - Named critical directives
  - Atomic directives
  - Simple locks

Q5: #pragma omp critical(name) 与 #pragma omp critical 有何区别？用处在哪里?

two blocks protected with critical directives with different names can be executed simultaneously.